



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Micromouse Maze Solver

Confronto tra Flood Fill e Incremental A*

Componenti del gruppo:

Kovacic Matteo — 736588

Mansueti Simone — 728046

Zambelli Samuele — 736800

Corso di Robotica

Anno Accademico 2025/2026

1 Introduzione

Il problema **Micromouse** consiste nel navigare autonomamente un labirinto 16×16 partendo senza una conoscenza pregressa della mappa. Il robot scopre i muri incrementalmente tramite sensori di prossimità e deve aggiornare la propria rappresentazione interna in tempo reale, bilanciando esplorazione e sfruttamento delle informazioni acquisite. Il problema rientra nella categoria degli ambienti parzialmente osservabili (POMDP), dove le decisioni devono essere prese sulla base di osservazioni locali incomplete.

Questo progetto implementa e confronta due approcci algoritmici distinti: **Flood Fill**, basato sulla propagazione BFS delle distanze, e **Incremental A***, basato su pianificazione euristica con replanning adattivo ispirato a LPA* e D* Lite.

2 Architettura del Sistema

Il sistema è organizzato in quattro livelli indipendenti:

Livello	Modulo	Responsabilità
I/O	<code>simulator_api.py</code>	Interfaccia stdin/stdout con il simulatore mms
Stato	<code>maze.py</code> , <code>robot.py</code>	Mappa interna a bitmask e posa del robot
Algoritmi	<code>flood_fill.py</code> , <code>incremental_astar.py</code>	Logica di navigazione
Valutazione	<code>metrics.py</code> , <code>benchmark.py</code> , <code>plotting.py</code>	Raccolta dati e analisi

`simulator_api.py` funge da layer **Adapter** tra il protocollo testuale del simulatore e il codice Python, esponendo funzioni di alto livello come `move_forward()` e `wall_front()`. `maze.py` rappresenta il labirinto tramite bitmask a 4 bit per cella per indicare le posizioni dei muri (Nord/Sud/Est/Ovest), aggiornata incrementalmente ad ogni nuova osservazione. `robot.py` traccia posizione, orientamento e storia delle celle visitate.

Ogni run è governato da una **macchina a stati a tre fasi** in `main.py`:

- **Fase 1 — Esplorazione:** il robot naviga da (0,0) al centro (7,7) scoprendo muri in tempo reale. Il percorso non è ottimo: è il primo percorso praticabile in condizioni di incertezza.
- **Fase 2 — Ritorno:** il goal viene invertito a (0,0). Il robot torna all'origine percorrendo zone diverse, accumulando ulteriore conoscenza del labirinto.
- **Fase 3 — Speed Run:** con la mappa parzialmente costruita, il sistema calcola il percorso più corto tra le sole celle visitate (`visited_only=True`) e lo esegue senza sensing.

3 Algoritmi di Navigazione

3.1 Flood Fill

Il Flood Fill assegna a ogni cella (x, y) una distanza $d(x, y)$ dal goal tramite BFS. Il robot segue sempre la cella adiacente con d minore. Quando viene scoperto un nuovo muro, tutte le distanze vengono ricalcolate sull'intera griglia — $O(W \times H)$ operazioni, con $W = H = 16$. L'algoritmo è **reattivo**: non mantiene un percorso esplicito ma segue localmente il gradiente. È completo (trova sempre il goal se esiste) e deterministico, caratteristiche che lo rendono particolarmente adatto alle competizioni reali.

3.2 Incremental A*

L'Incremental A* pianifica un percorso esplicito minimizzando $f(x, y) = g(x, y) + h(x, y)$, dove g è il costo percorso finora e h è la distanza di Manhattan al goal. L'euristica è **ammissibile** (non sovrastima mai) e **consistente** (monotona), garantendo ottimalità con peso $w = 1$. La componente incrementale consente di validare il percorso corrente ad ogni nuovo muro e ricalcolare solo la porzione invalidata, invece di ripartire da zero. La ricerca utilizza un min-heap binario con complessità $O(k \log k)$ per replan, dove $k \ll 256$ è la regione invalidata.

3.3 Visualizzazione nel Simulatore

Il simulatore mms aggiorna i colori delle celle in tempo reale, riflettendo il ragionamento interno dell'algoritmo ad ogni passo.

Tabella 1: Legenda dei colori utilizzati durante la simulazione.

Algoritmo	Fasi 1–2 (esplorazione)		Fase 3 (speed run)	
Flood Fill	Verde	goal ($d = 0$)	Ciano	celle visitate
	Giallo	$1 \leq d \leq 4$	Verde	percorso ottimale
	Ciano	$5 \leq d \leq 14$		
	Blu	$d \geq 15$		
	Rosso	$d = \infty$ (irraggiungibile)		
Incremental A*	Verde	goal	Ciano	celle visitate nelle fasi 1–2
	Giallo	frontiera di ricerca (f visibile)	Verde	percorso ottimale
	Ciano	celle esplorate		
	Nero	celle inesplorate		

4 Risultati Sperimentali

La valutazione sperimentale è stata condotta utilizzando un benchmark costituito da cinque differenti tipologie di labirinto: *competition*, *dead_end*, *multiple_path*, *open_area* e *symmetric*. Complessivamente, sono state effettuate 40 esecuzioni, equamente ripartite tra i due algoritmi oggetto di studio (20 run per algoritmo). I risultati ottenuti mostrano che entrambi gli algoritmi hanno raggiunto il goal nel 100% delle esecuzioni effettuate, evidenziando un elevato livello di robustezza e affidabilità rispetto a tutte le configurazioni ambientali considerate.

Gli esperimenti sono stati eseguiti in un ambiente di simulazione basato su Windows 11 e Python 3.11.9, su una piattaforma hardware equipaggiata con processore AMD Ryzen 7 5700X. Per la simulazione è stato utilizzato il simulatore Micromouse MMS sviluppato da Mackorone (<https://github.com/mackorone/mms>). Durante l'intera campagna sperimentale, la velocità di esecuzione del simulatore è stata impostata al valore massimo disponibile e mantenuta costante in tutte le prove, al fine di garantire uniformità nelle condizioni di valutazione.

4.1 Confronto Generale tra gli Algoritmi

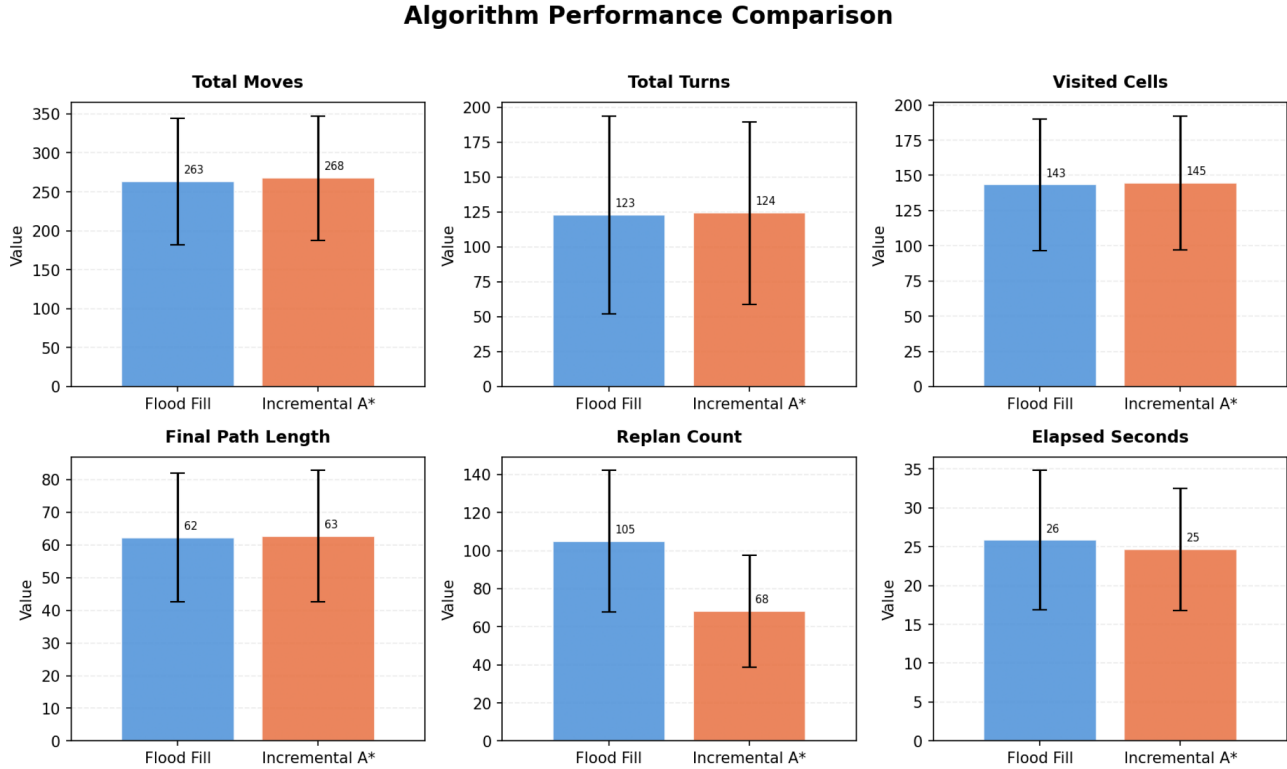


Figura 1: Confronto tra Flood Fill e Incremental A* sui principali indicatori di performance.

La Figura 1 mostra il confronto aggregato sui sei indicatori principali. I risultati evidenziano una sostanziale equivalenza tra i due approcci: le mosse totali sono 263 per Flood Fill e 268 per A*, le celle visitate 143 contro 145, e il percorso finale della speed run praticamente identico (62 vs 63 passi). La differenza più marcata riguarda il **replan count**: il Flood Fill esegue in media 105 replan contro i 68 dell'A*. Questo è atteso: il Flood Fill ricalcola l'intera griglia ad ogni muro scoperto, mentre l'A* invalida e ricalcola solo la porzione di percorso interessata. Il tempo di esecuzione è leggermente superiore per Flood Fill (26s vs 25s), coerentemente con il maggior numero di ricalcoli globali.

4.2 Sensibilità alla Tipologia di Labirinto

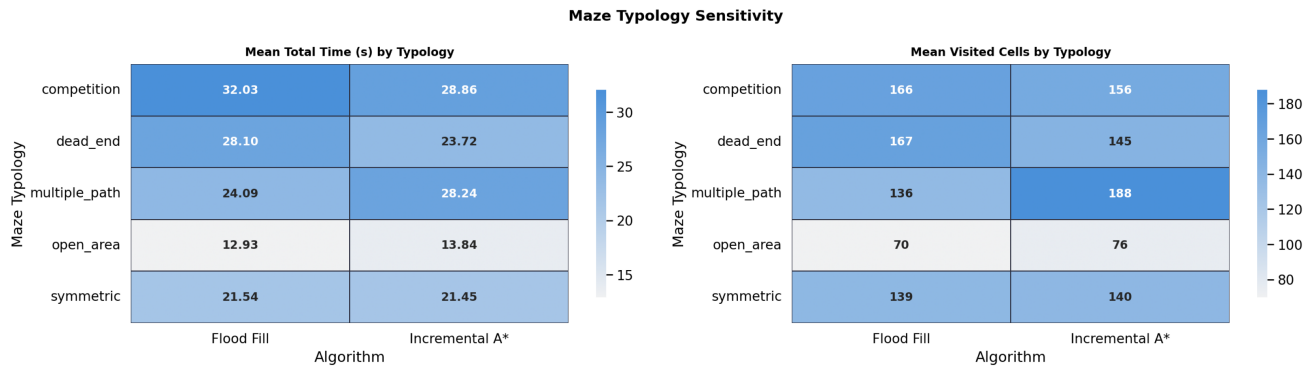


Figura 2: Heatmap della sensibilità degli algoritmi alle diverse tipologie di labirinto: tempo medio (sinistra) e celle visitate (destra).

La Figura 2 rivela come le prestazioni varino significativamente tra le tipologie. I labirinti **open_area** sono i più semplici per entrambi (FF: 12.93s, A*: 13.84s; celle visitate 70 e 76), mentre i labirinti **competition** sono i più onerosi (FF: 32.03s, A*: 28.86s). La differenza più interessante emerge nei labirinti **multiple_path**: il Flood Fill visita 136 celle contro le 188 dell'A*, suggerendo che la presenza di percorsi alternativi confonde la pianificazione euristica dell'A*, che tende a seguire un percorso esplicito e a doverlo ricalcolare più volte quando trova muri su percorsi alternativi. Nei labirinti **dead_end** invece l'A* è più efficiente (145 celle vs 167), probabilmente perché il replanning locale evita di riesaminare vicoli ciechi già scartati.

4.3 Analisi della Fase 2

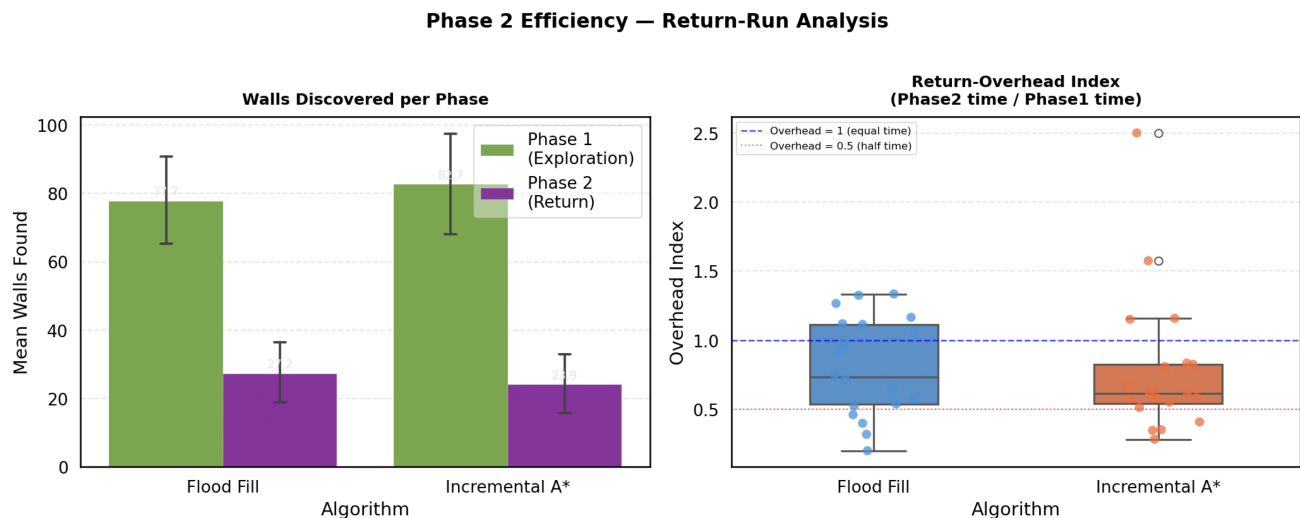


Figura 3: Analisi della Fase 2: muri scoperti per fase (sinistra) e Return-Overhead Index (destra).

La Figura 3 analizza l'efficacia della fase di ritorno. Il grafico di sinistra mostra che entrambi gli algoritmi scoprono la maggior parte dei muri in Fase 1 (FF: 78, A*: 83) e una quota minore in Fase 2 (FF: 27, A*: 24). Il **Return-Overhead Index** (rapporto tra tempo Fase 2 e tempo Fase 1) è in mediana intorno a 0.7

per entrambi — la fase di ritorno costa circa il 70% del tempo dell’esplorazione iniziale. L’A* mostra una distribuzione più concentrata e meno variabile, mentre il Flood Fill presenta alcuni outlier con overhead superiore a 1 (casi in cui il ritorno è costato più dell’andata), probabilmente su labirinti con struttura asimmetrica che penalizza maggiormente il percorso inverso.

4.4 Return on Investment dell’Esplorazione

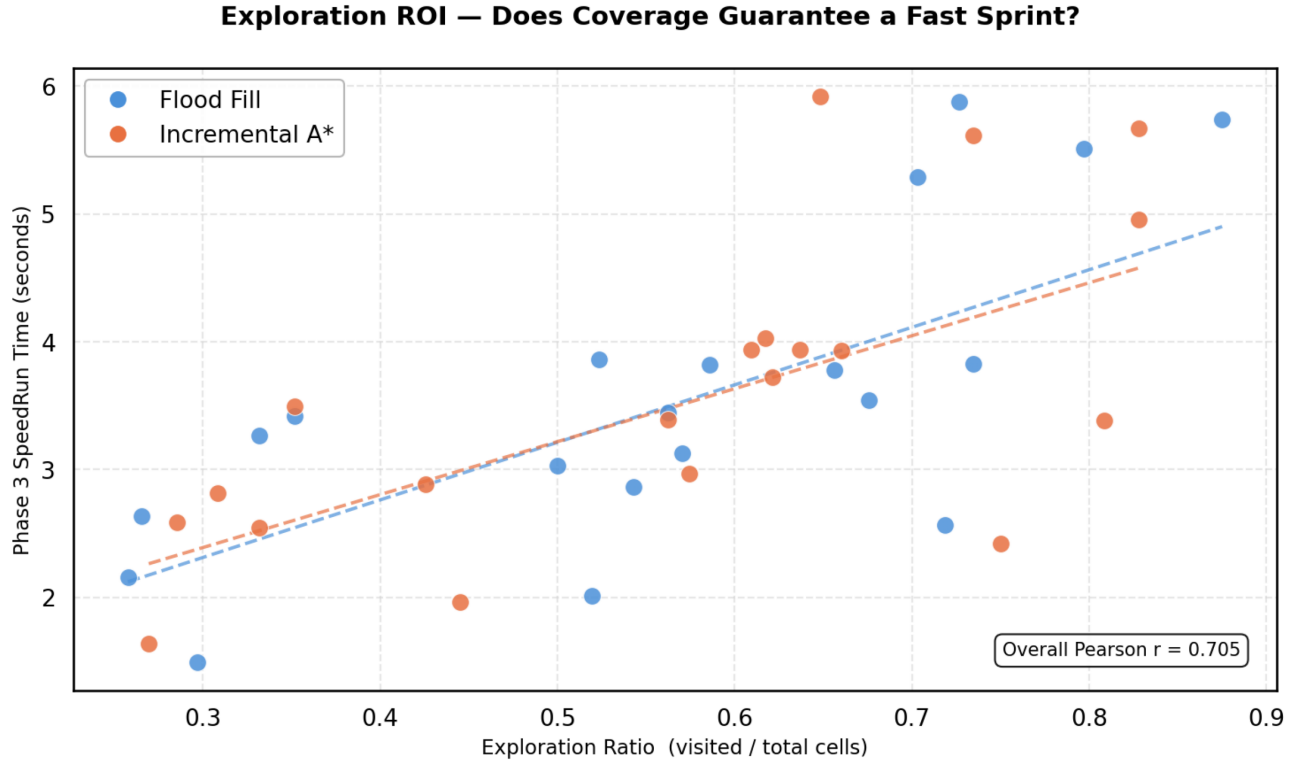


Figura 4: Exploration ROI: correlazione tra copertura del labirinto e tempo della speed run.

La Figura 4 affronta una domanda fondamentale: esplorare più celle garantisce una speed run più veloce? Il grafico mostra sull’asse X il rapporto di copertura (celle visitate / celle totali) e sull’asse Y il tempo della Fase 3. La correlazione positiva è evidente (Pearson $r = 0.705$): più il robot esplora, più la speed run è lunga. Questo risultato è **controintuitivo** a prima vista, ma ha una spiegazione logica: i labirinti più complessi (competition, dead_end) richiedono più esplorazione e producono naturalmente percorsi finali più lunghi, mentre i labirinti semplici (open_area) si esplorano poco e si percorrono in tempi brevi. Non è quindi la copertura a causare percorsi più lunghi, ma la complessità del labirinto a determinare entrambe le variabili. I due algoritmi mostrano linee di tendenza quasi sovrapposte, confermando la loro equivalenza in termini di ROI esplorativo.

5 Discussione

5.1 Effetto della Revisit Penalty

In questa sezione analizziamo l’impatto della revisit penalty.

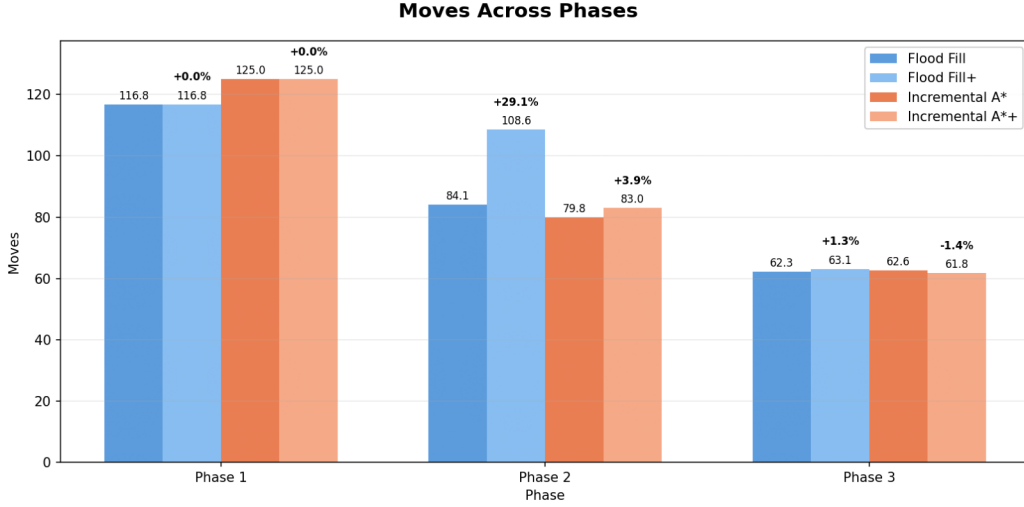


Figura 5: Effetto della revisit penalty rispetto alla configurazione base.

La Figura 5 mostra l'effetto della revisit penalty sul numero medio di mosse nelle tre fasi dell'esplorazione. Le varianti “+” indicano la versione con penalty attiva. L'effetto è nettamente più marcato per il **Flood Fill**, che nella Fase 2 richiede il 29.1% di mosse in più, mentre nelle altre fasi la variazione è trascurabile. Per l'**Incremental A***, l'impatto rimane contenuto in tutte le fasi (+3.9% nella Fase 2 e -1.4% nella Fase 3). Questo suggerisce che l'euristica dell'A* riduce già naturalmente i ritorni su celle visitate, mentre il Flood Fill risulta più sensibile all'introduzione di costi aggiuntivi per il revisiting.

5.2 Smoothness Cinematica

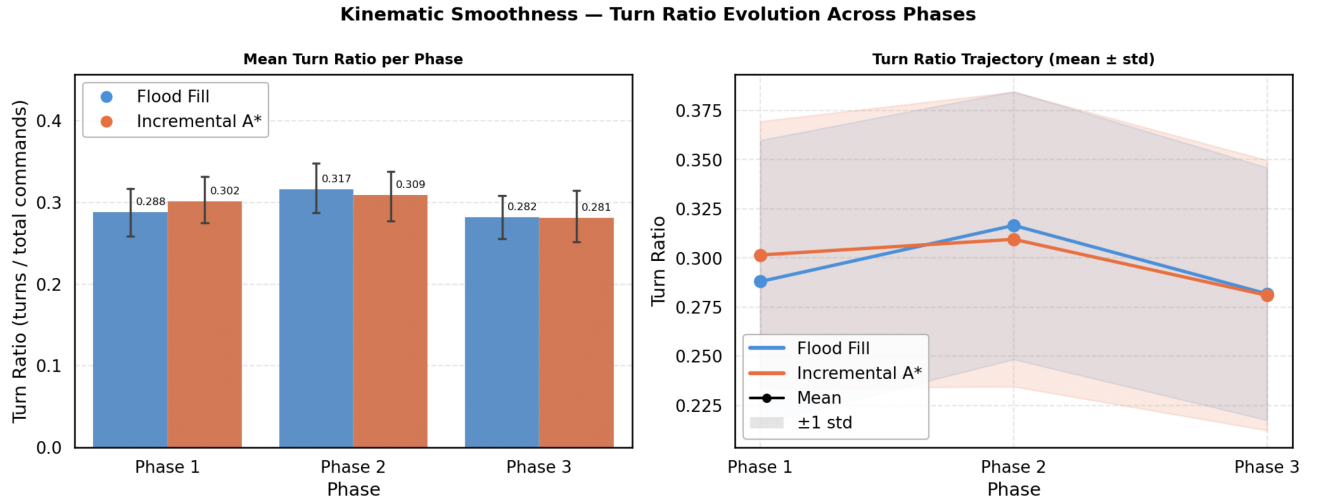


Figura 6: Evoluzione del turn ratio nelle tre fasi: rapporto svolte/comandi totali.

La Figura 6 analizza la **smoothness cinematica** tramite il turn ratio (svolte / comandi totali). Un valore basso indica un percorso più diretto e lineare, più efficiente su hardware reale dove le svolte hanno un costo energetico e temporale superiore alle mosse lineari.

Entrambi gli algoritmi mostrano un pattern analogo nelle tre fasi: turn ratio simile in Fase 1 (FF: 0.288, A*: 0.302), leggermente più alto in Fase 2 (FF: 0.317, A*: 0.309), e minimo in Fase 3 (FF: 0.282, A*:

0.281). L'aumento in Fase 2 è atteso: il ritorno verso l'origine segue un percorso meno diretto rispetto all'esplorazione iniziale, con più cambi di direzione. La Fase 3 produce i percorsi più fluidi perché calcolati sulla mappa più completa disponibile. La quasi identità dei valori tra i due algoritmi in tutte le fasi conferma che, dal punto di vista cinematico, i due approcci producono traiettorie di qualità equivalente.

5.3 Considerazioni Generali

I risultati sperimentali confermano che i due algoritmi sono sostanzialmente equivalenti nelle condizioni testate, con differenze statisticamente modeste sugli indicatori globali. Le divergenze emergono a livello di tipologia di labirinto, suggerendo che la scelta dell'algoritmo ottimale dipende dalla struttura dell'ambiente:

- **Labirinti open area e symmetric:** equivalenti, con lieve vantaggio del Flood Fill per minor numero di celle visitate
- **Labirinti competition:** A* leggermente migliore in termini di tempo (28.86s vs 32.03s)
- **Labirinti multiple_path:** Flood Fill nettamente più efficiente (136 vs 188 celle visitate)
- **Labirinti dead_end:** A* più efficiente (145 vs 167 celle visitate)

La revisit penalty risulta vantaggiosa solo per il Flood Fill, mentre per l'A* introduce overhead senza benefici rilevanti.

6 Conclusioni

Il progetto ha implementato e confrontato con successo due algoritmi di navigazione autonoma per il problema Micromouse in ambiente parzialmente osservabile. Entrambi gli algoritmi raggiungono il goal nel 100% dei run su tutte le tipologie di labirinto testate, dimostrando robustezza e completezza algoritmiche.

La principale conclusione è che **non esiste un algoritmo universalmente superiore**. Il Flood Fill si dimostra competitivo rispetto all'A* e lo supera su labirinti con percorsi multipli, mentre l'Incremental A* è preferibile su labirinti strutturati e con vicoli ciechi, dove il replanning locale evita ricalcoli globali. In entrambi i casi la qualità del percorso finale è praticamente identica: le differenze riguardano l'efficienza esplorativa, non la soluzione trovata. L'analisi della revisit penalty mostra effetti asimmetrici sui due approcci, aprendo la strada a configurazioni ibride.

Per sviluppi futuri si individuano le seguenti direzioni:

- **Esplorazione attiva:** massimizzare la copertura prima della speed run per ridurre l'incertezza sul percorso ottimo
- **Algoritmi ibridi:** combinare il gradiente globale del Flood Fill con l'euristica dell'A* per ottenere il meglio di entrambi
- **Simulazione più dettagliata:** migliorare il realismo del contesto simulativo introducendo movimenti più fini (ad esempio rotazioni a 45° e spostamenti a mezzo step) oppure estendendo il simulatore tramite API per supportare dinamiche di velocità variabile durante l'esecuzione
- **Hardware reale:** portare il sistema su un robot fisico, aggiungendo gestione dell'odometria e incertezza sensoriale